In [50]:
```
#import libraries
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
from plotly.offline import iplot
import plotly as py
import plotly.tools as tls
import cufflinks as cf
%matplotlib inline
```

Load and View Dataset

In [51]:
```
inventory_path="C:/Users/nairs/Downloads/Mexico Toy Sales Dataset/inventory.csv"
products_path="C:/Users/nairs/Downloads/Mexico Toy Sales Dataset/products.csv"
sales_path="C:/Users/nairs/Downloads/Mexico Toy Sales Dataset/sales.csv"
stores_path="C:/Users/nairs/Downloads/Mexico Toy Sales Dataset/stores.csv"
```

In [52]:
```
inventory_df=pd.read_csv(inventory_path,encoding='ISO-8859-1')
products_df=pd.read_csv(products_path,encoding='ISO-8859-1')
sales_df=pd.read_csv(sales_path,encoding='ISO-8859-1')
stores_df=pd.read_csv(stores_path,encoding='ISO-8859-1')
```

In [53]:
```
inventory_df.head()
```

Out[53]:

	Store_ID	Product_ID	Stock_On_Hand
0	1	1	27
1	1	2	0
2	1	3	32
3	1	4	6
4	1	5	0

In [54]:
```
products_df.head()
```

Out[54]:

	Product_ID	Product_Name	Product_Category	Product_Cost	Product_Price
0	1	Action Figure	Toys	$9.99	$15.99
1	2	Animal Figures	Toys	$9.99	$12.99
2	3	Barrel O' Slime	Art & Crafts	$1.99	$3.99
3	4	Chutes & Ladders	Games	$9.99	$12.99
4	5	Classic Dominoes	Games	$7.99	$9.99

In [55]:
```
sales_df.head()
```

Out[55]:

	Sale_ID	Date	Store_ID	Product_ID	Units
0	1	2017-01-01	24	4	1
1	2	2017-01-01	28	1	1
2	3	2017-01-01	6	8	1
3	4	2017-01-01	48	7	1
4	5	2017-01-01	44	18	1

In [56]:
```
stores_df.head()
```

Out[56]:

	Store_ID	Store_Name	Store_City	Store_Location	Store_Open_Date
0	1	Maven Toys Guadalajara 1	Guadalajara	Residential	1992-09-18
1	2	Maven Toys Monterrey 1	Monterrey	Residential	1995-04-27
2	3	Maven Toys Guadalajara 2	Guadalajara	Commercial	1999-12-27
3	4	Maven Toys Saltillo 1	Saltillo	Downtown	2000-01-01
4	5	Maven Toys La Paz 1	La Paz	Downtown	2001-05-31

Merge Datasets

In [57]:
```
merged_df=sales_df.merge(products_df,on="Product_ID",how="left")
merged_df=merged_df.merge(stores_df,on="Store_ID",how="left")
merged_df=merged_df.merge(inventory_df,on=["Store_ID","Product_ID"],how="left")
```

In [58]:
```
merged_df.head()
```

Out[58]:

	Sale_ID	Date	Store_ID	Product_ID	Units	Product_Name	Product_Category	Prodı
0	1	2017-01-01	24	4	1	Chutes & Ladders	Games	
1	2	2017-01-01	28	1	1	Action Figure	Toys	
2	3	2017-01-01	6	8	1	Deck Of Cards	Games	
3	4	2017-01-01	48	7	1	Dart Gun	Sports & Outdoors	
4	5	2017-01-01	44	18	1	Lego Bricks	Toys	

In [59]:
```
merged_df.tail()
```

Out[59]:

	Sale_ID	Date	Store_ID	Product_ID	Units	Product_Name	Product_Category
829257	829258	2018-09-30	24	19	1	Magic Sand	Art & Crafts
829258	829259	2018-09-30	16	35	1	Uno Card Game	Games
829259	829260	2018-09-30	22	19	1	Magic Sand	Art & Crafts
829260	829261	2018-09-30	13	2	2	Animal Figures	Toys
829261	829262	2018-09-30	32	6	1	Colorbuds	Electronics

Check Shape and columns

In [60]:
```
print("Rows:",merged_df.shape[0])
print('Columns:',
merged_df.shape[1])
```

```
Rows: 829262
Columns: 14
```

In [61]:
```
# Dataset Rows & Column count
rows,cols=merged_df.shape
print(f'There are {rows} rows and {cols} columns in the dataset.')
```

```
There are 829262 rows and 14 columns in the dataset.
```

Dataset Information

In [62]:
```
merged_df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 829262 entries, 0 to 829261
Data columns (total 14 columns):
 #   Column            Non-Null Count   Dtype
---  ------            --------------   -----
 0   Sale_ID           829262 non-null  int64
 1   Date              829262 non-null  object
 2   Store_ID          829262 non-null  int64
 3   Product_ID        829262 non-null  int64
 4   Units             829262 non-null  int64
 5   Product_Name      829262 non-null  object
 6   Product_Category  829262 non-null  object
 7   Product_Cost      829262 non-null  object
 8   Product_Price     829262 non-null  object
 9   Store_Name        829262 non-null  object
 10  Store_City        829262 non-null  object
 11  Store_Location    829262 non-null  object
 12  Store_Open_Date   829262 non-null  object
 13  Stock_On_Hand     823030 non-null  float64
dtypes: float64(1), int64(4), object(9)
memory usage: 88.6+ MB
```

Data Types

In [63]:
```
merged_df.dtypes
```

Out[63]:
```
Sale_ID              int64
Date                object
Store_ID             int64
Product_ID           int64
Units                int64
Product_Name        object
Product_Category    object
Product_Cost        object
Product_Price       object
Store_Name          object
Store_City          object
Store_Location      object
Store_Open_Date     object
Stock_On_Hand      float64
dtype: object
```

Duplicate Records

In [64]:
```
# Datset Duplicate Values Count
duplicate_rows=merged_df.duplicated().sum()
print(f'There are {duplicate_rows} duplicate_rows in the dataset')
```

```
There are 0 duplicate_rows in the dataset
```

Missing Values

In [65]:
```
missing_values=merged_df.isnull().sum()
print(missing_values)
```

```
Sale_ID               0
Date                  0
Store_ID              0
Product_ID            0
Units                 0
Product_Name          0
Product_Category      0
Product_Cost          0
Product_Price         0
Store_Name            0
Store_City            0
Store_Location        0
Store_Open_Date       0
Stock_On_Hand      6232
dtype: int64
```

In [66]:
```
merged_df["Stock_On_Hand"]=merged_df["Stock_On_Hand"].fillna(0)
```

In [67]:
```
missing_values=merged_df.isnull().sum()
print(missing_values)
```

```
Sale_ID            0
Date               0
Store_ID           0
Product_ID         0
Units              0
Product_Name       0
Product_Category   0
Product_Cost       0
Product_Price      0
Store_Name         0
Store_City         0
Store_Location     0
Store_Open_Date    0
Stock_On_Hand      0
dtype: int64
```

Date Conversion

In [68]:
```
merged_df["Date"]=pd.to_datetime(merged_df["Date"])
merged_df["Year"]=merged_df["Date"].dt.year
merged_df["Month"]=merged_df["Date"].dt.month_name()
merged_df["Day"]=merged_df["Date"].dt.day_name()
```

Conversion of Data types

In [69]:
```
merged_df["Product_Cost"]=(merged_df["Product_Cost"].astype(str).str.replace(r"[
merged_df["Product_Price"]=(merged_df["Product_Price"].astype(str).str.replace(r
merged_df["Product_Cost"]=pd.to_numeric(merged_df["Product_Cost"],errors="coerce
merged_df["Product_Price"]=pd.to_numeric(merged_df["Product_Price"],errors="coer
```

In [70]:
```
merged_df["Revenue"]=merged_df["Units"]*merged_df["Product_Price"]
merged_df["Cost"]=merged_df["Units"]*merged_df["Product_Cost"]
merged_df["Profit"]=merged_df["Revenue"]-merged_df["Cost"]
merged_df["Profit_Margin"]=(merged_df["Profit"]/merged_df["Revenue"]*100)
```

In [71]:
```
merged_df[["Units","Product_Cost","Product_Price"]].dtypes
```

Out[71]:
```
Units             int64
Product_Cost    float64
Product_Price   float64
dtype: object
```

In [72]:
```
merged_df.head()
```

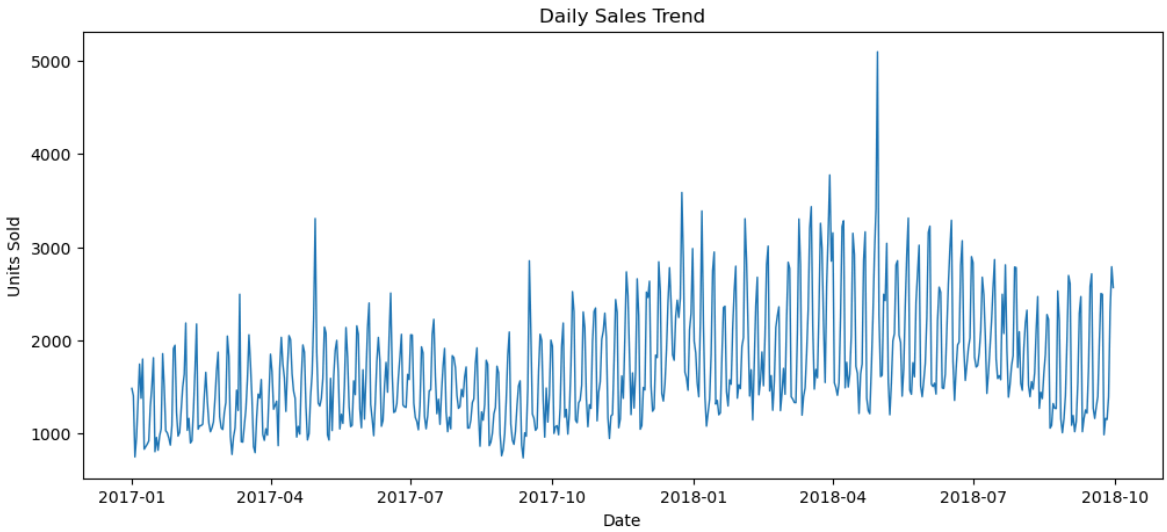
Out[72]:

	Sale_ID	Date	Store_ID	Product_ID	Units	Product_Name	Product_Category	Prodı
0	1	2017-01-01	24	4	1	Chutes & Ladders	Games	
1	2	2017-01-01	28	1	1	Action Figure	Toys	
2	3	2017-01-01	6	8	1	Deck Of Cards	Games	
3	4	2017-01-01	48	7	1	Dart Gun	Sports & Outdoors	
4	5	2017-01-01	44	18	1	Lego Bricks	Toys	

5 rows × 21 columns

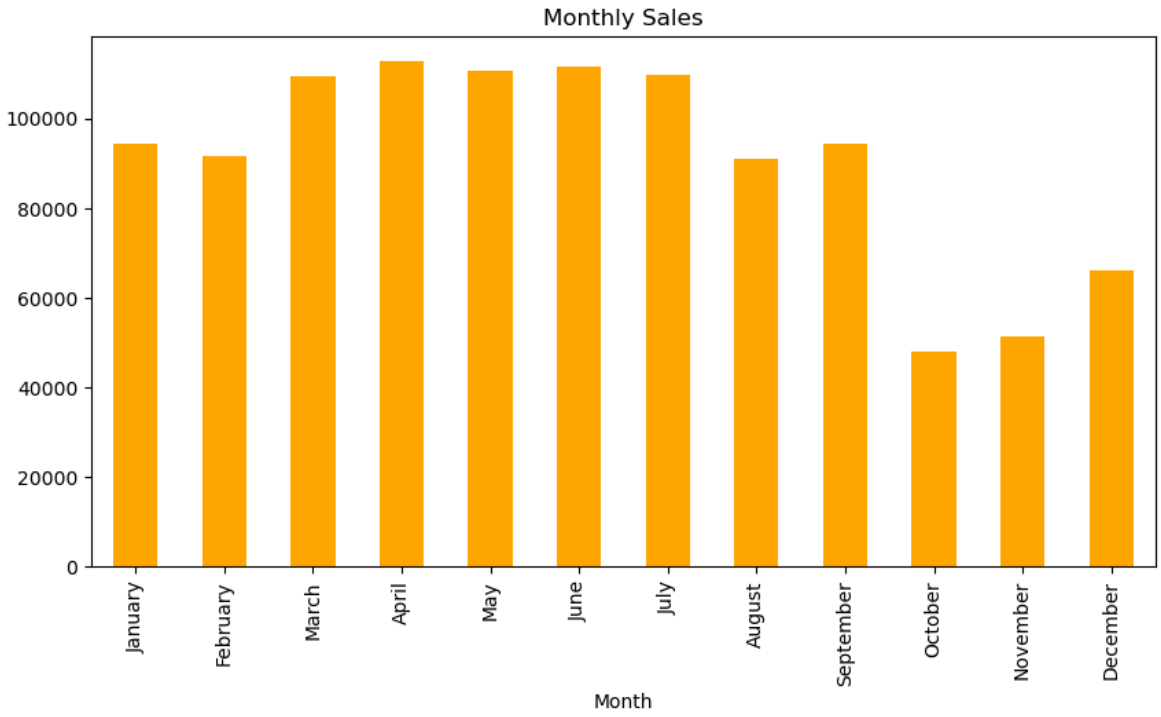
1. Sales Trend Over Time(Line Chart)

In [73]:
```
sales_trend=merged_df.groupby("Date")["Units"].sum()
plt.figure(figsize=(12,5))
plt.plot(sales_trend.index,sales_trend.values,linewidth=1)
plt.title("Daily Sales Trend")
plt.xlabel("Date")
plt.ylabel("Units Sold")
plt.show()
```



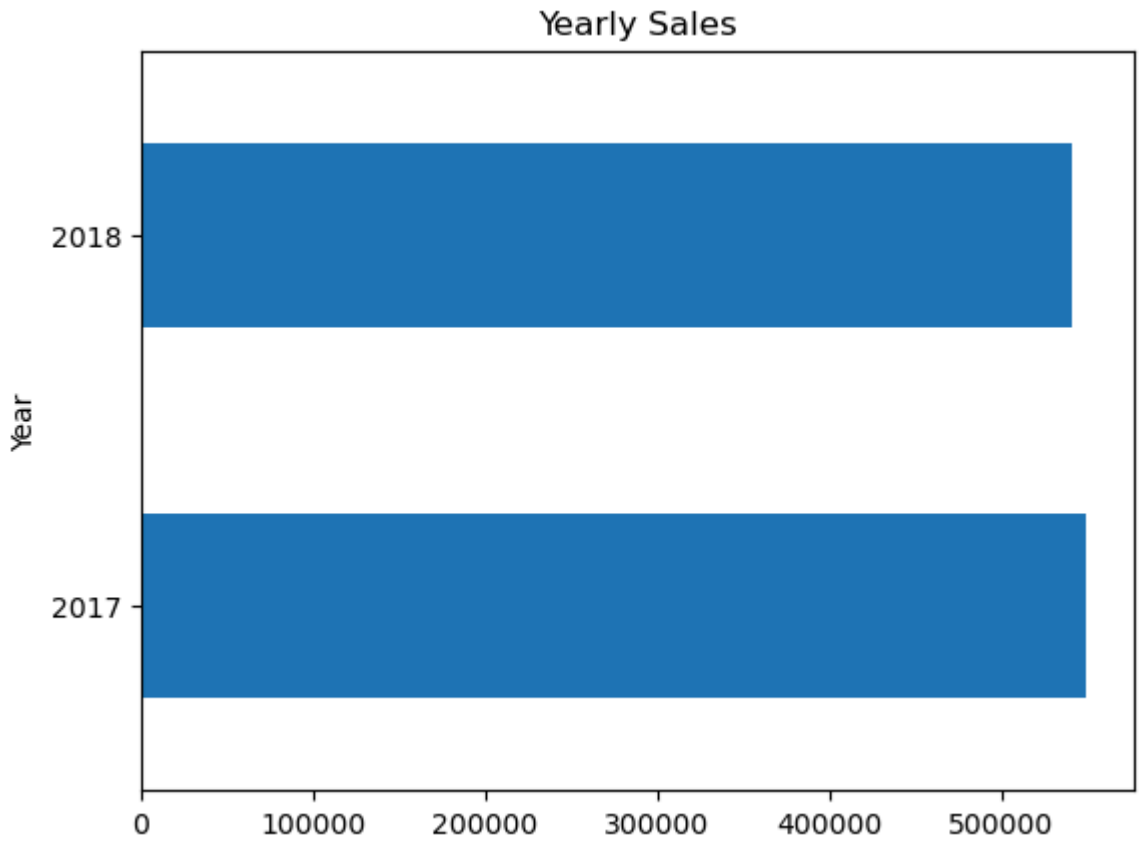
2.Monthly Sales(Bar Chart)

In [74]:
```
month_order=["January","February","March","April","May","June","July","August","
monthly=merged_df.groupby("Month")["Units"].sum().reindex(month_order)
monthly.plot(kind="bar",color="orange",figsize=(10,5))
plt.title("Monthly Sales")
plt.show()
```



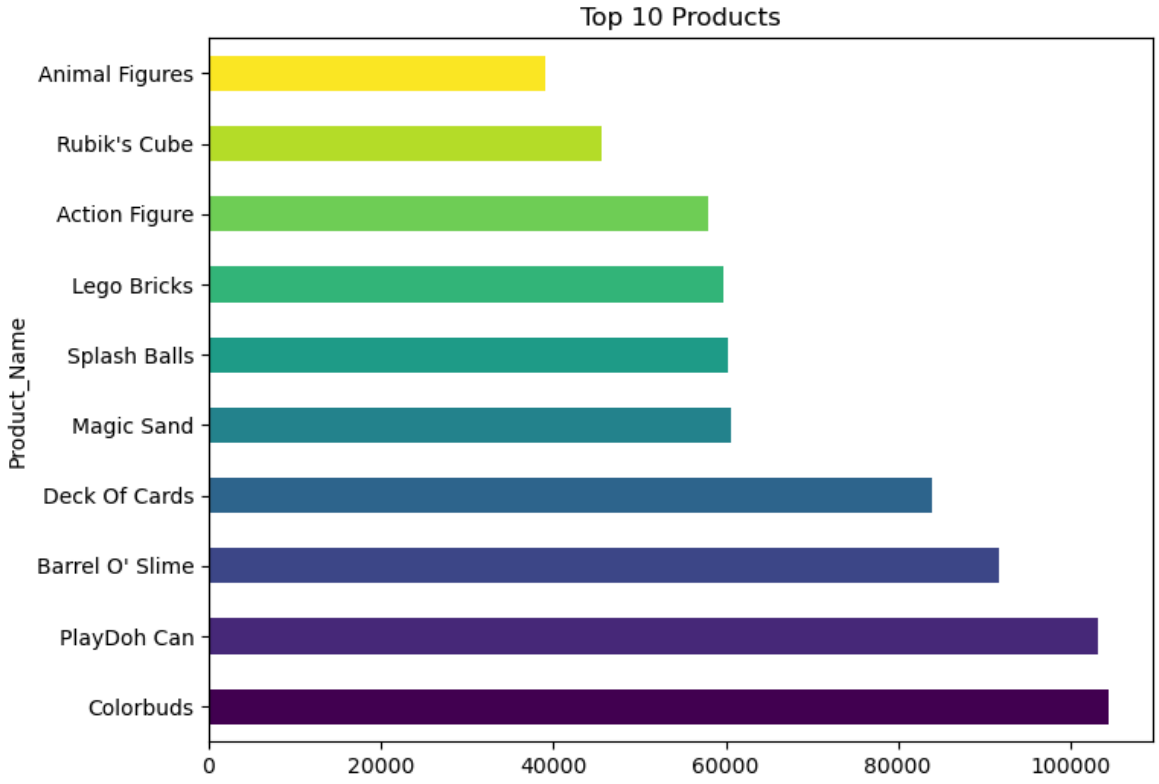
3.Yearly Sales

In [75]:
```
merged_df.groupby("Year")["Units"].sum().plot(kind="barh")
plt.title("Yearly Sales")
plt.show()
```



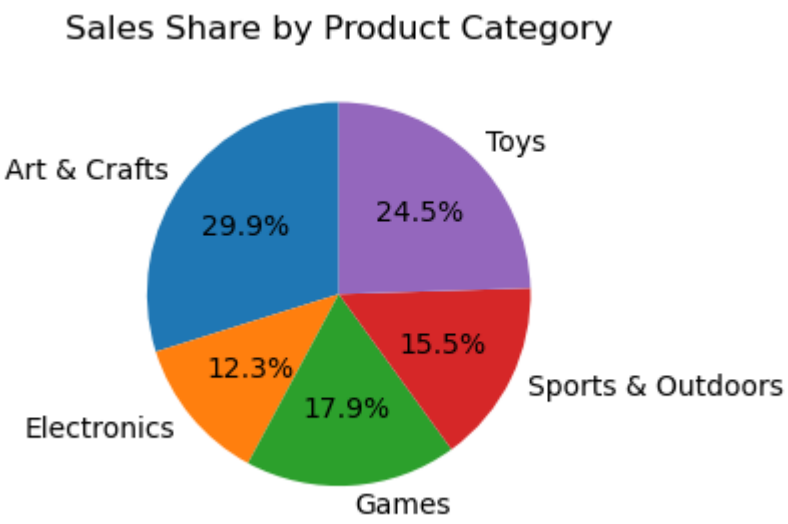
4. Top 10 Products by Units(Horizontal Bar)

In [76]:
```
top_products=merged_df.groupby("Product_Name")["Units"].sum().nlargest(10)
colors=plt.cm.viridis(np.linspace(0,1,len(top_products)))
top_products.plot(kind="barh",figsize=(8,6),color=colors)
plt.title("Top 10 Products")
plt.show()
```



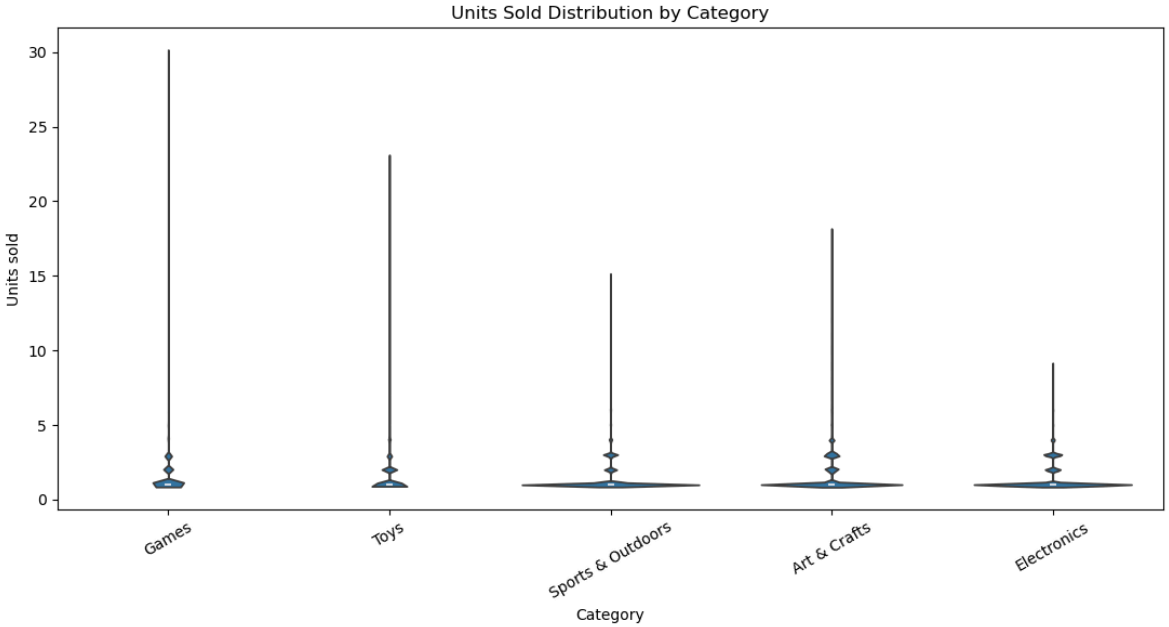
5.Sales Share by Category(Pie Chart)

In [77]:
```
category_sales=(merged_df.groupby("Product_Category")['Units'].sum())
plt.figure(figsize=(4,4))
plt.pie(category_sales.values,
        labels=category_sales.index,
        autopct="%1.1f%%",
        startangle=90
        )
plt.title("Sales Share by Product Category")
plt.tight_layout()
plt.show()
```



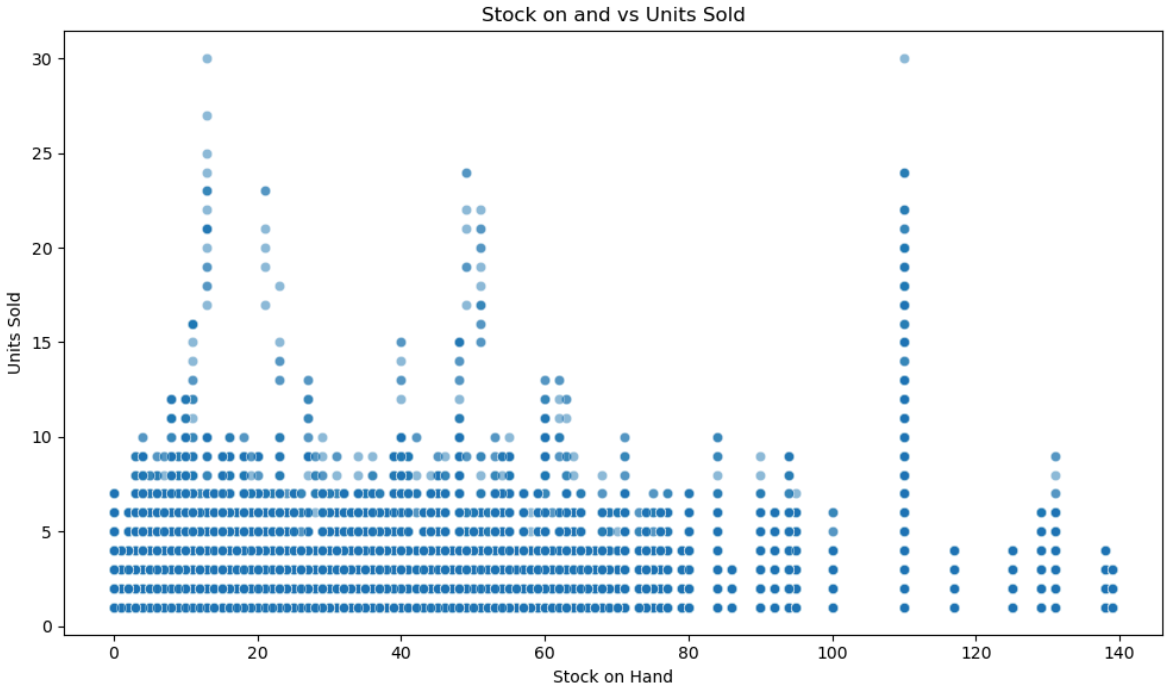
6.Units Sold by Category-Violin Plot

In [78]:
```
plt.figure(figsize=(11,6))
sns.violinplot(data=merged_df,
               x="Product_Category",
               y="Units")
plt.title("Units Sold Distribution by Category")
plt.xlabel("Category")
plt.ylabel("Units sold")
plt.xticks(rotation=30)
plt.tight_layout()
plt.show()
```



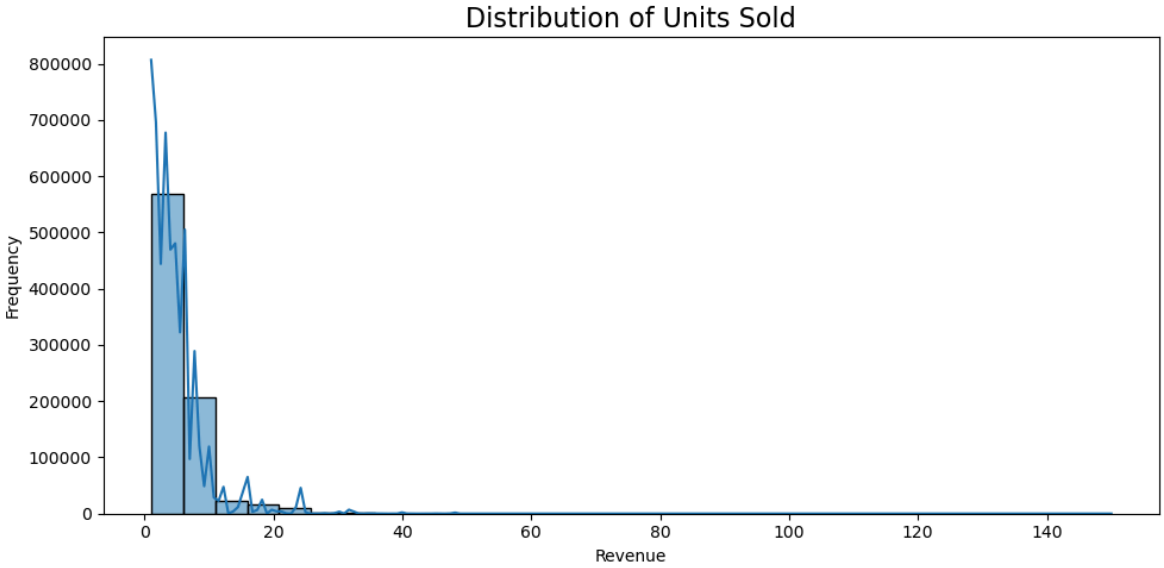
7.Stock on Hand vs Units Sold

In [79]:
```
plt.figure(figsize=(10,6))
sns.scatterplot(data=merged_df,
                x="Stock_On_Hand",
                y="Units",
                alpha=0.5)
plt.title("Stock on and vs Units Sold")
plt.xlabel("Stock on Hand")
plt.ylabel("Units Sold")
plt.tight_layout()
plt.show()
```



8.Units Sold Distribution

In [80]:
```
plt.figure(figsize=(10,5))
sns.histplot(merged_df["Profit"],bins=30,kde=True)
plt.title("Distribution of Units Sold",fontsize=16)
plt.xlabel("Revenue")
plt.ylabel("Frequency")
plt.tight_layout()
plt.show()
```

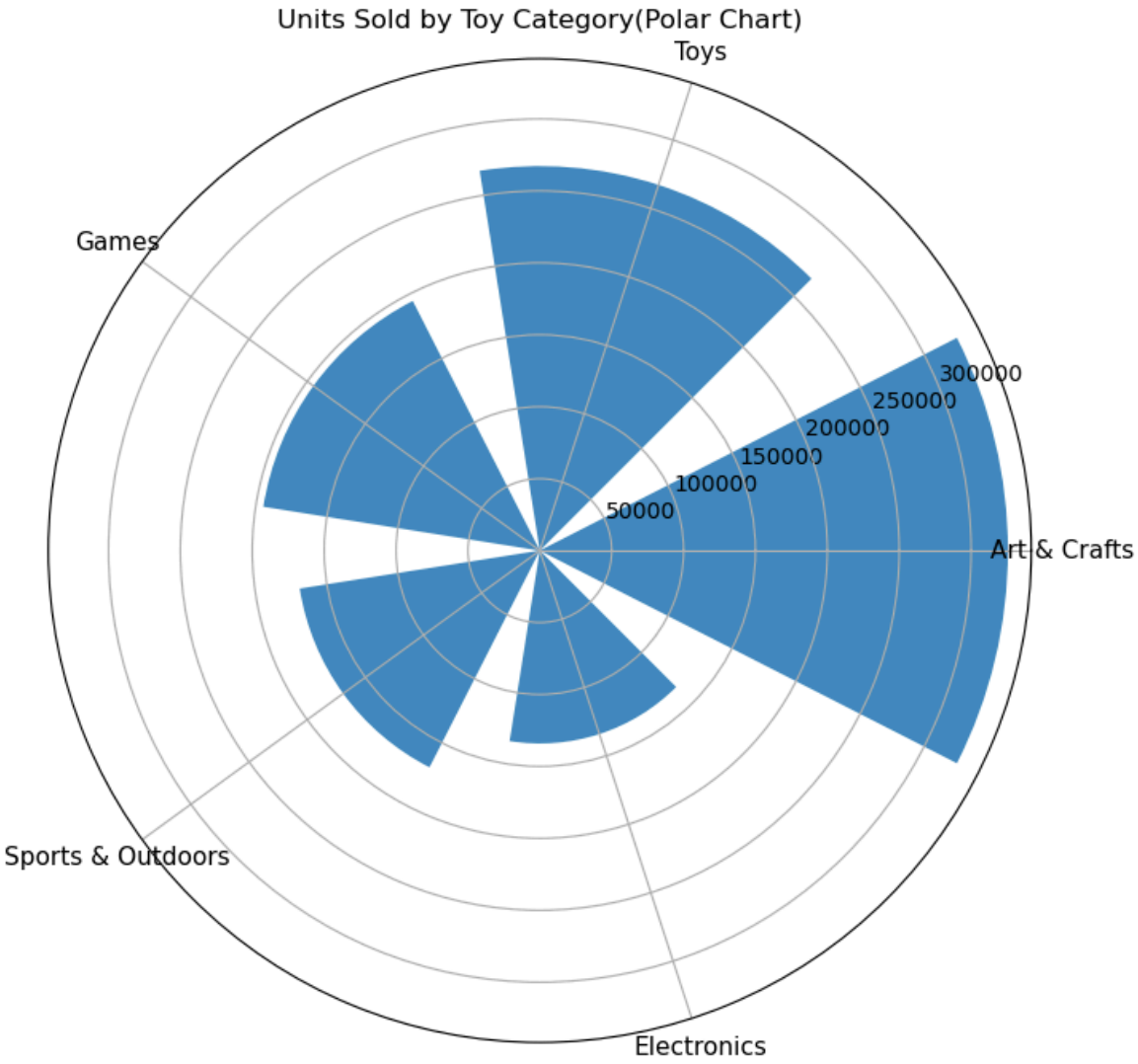


9.Sales by Category(Polar Bar Chart)

In [81]:
```
polar_df=(merged_df.groupby('Product_Category')['Units'].sum().sort_values(ascen
categories=polar_df.index
values=polar_df.values

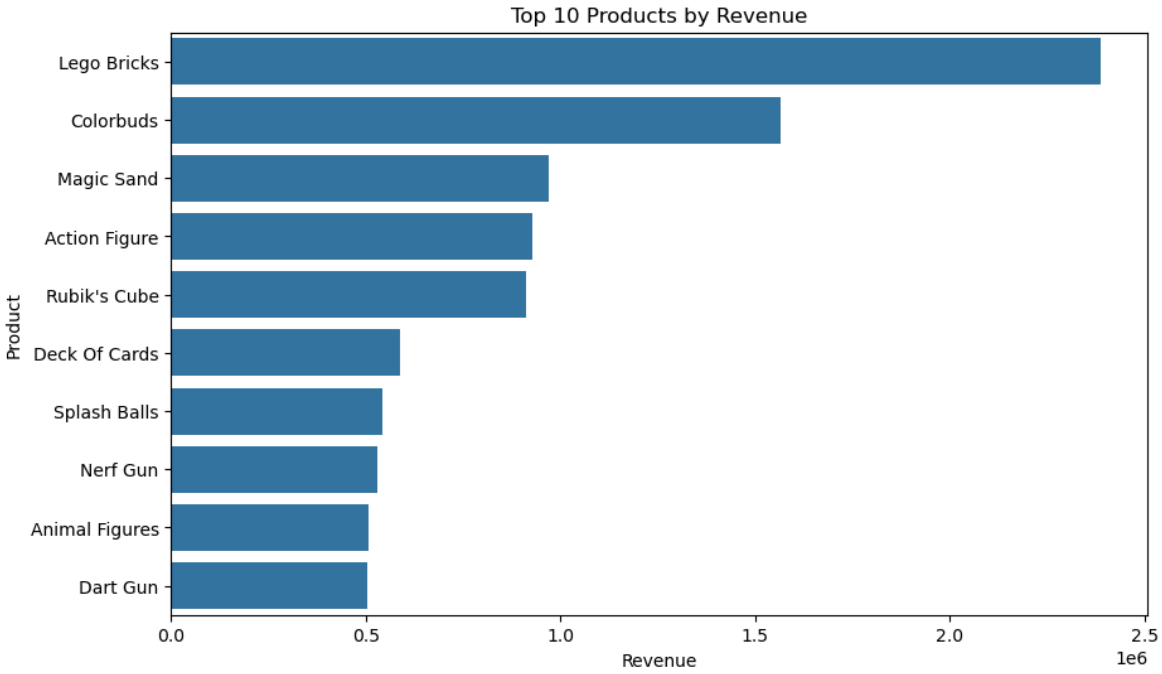
angles=np.linspace(0,2*np.pi,len(categories),endpoint=False)
width=2*np.pi/len(categories)*0.75
fig,ax=plt.subplots(figsize=(8,8),
                    subplot_kw={'projection':'polar'})
bars=ax.bar(angles,values,width=width,alpha=0.85)

ax.set_xticks(angles)
ax.set_xticklabels(categories,fontsize=11)
ax.set_title("Units Sold by Toy Category(Polar Chart)")
plt.show()
```



10.Top 10 Products by Revenue

In [82]:
```
top_products=(merged_df.groupby("Product_Name")["Revenue"].sum().sort_values(asc
plt.figure(figsize=(10,6))
sns.barplot(
                  x=top_products.values,
                  y=top_products.index
              )
plt.title("Top 10 Products by Revenue")
plt.xlabel("Revenue")
plt.ylabel("Product")
plt.show()
```



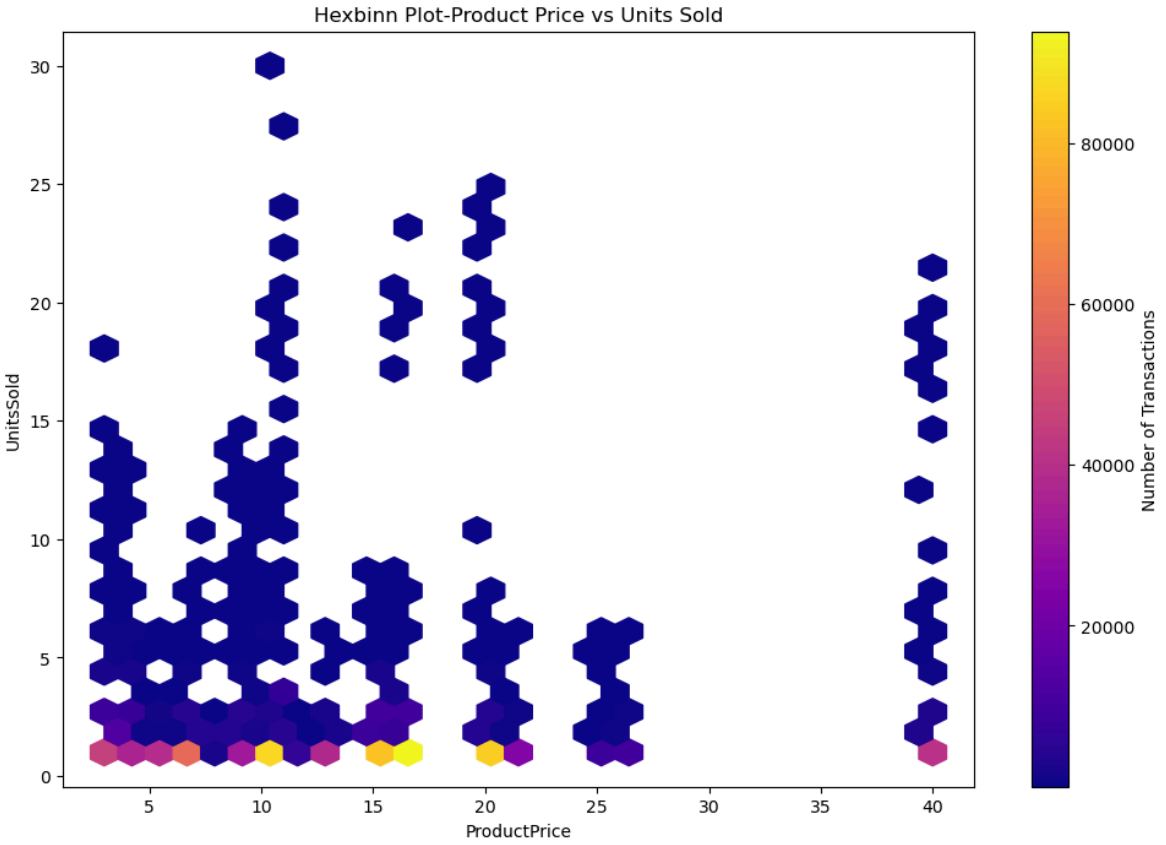
11.Treemap-Store-Category Revenue

In [83]:
```
import plotly.express as px

tree_df=merged_df.groupby(['Store_Name','Product_Category'],as_index=False)["Rev
fig=px.treemap(tree_df,path=["Store_Name","Product_Category"],values="Revenue",c
               title="Treemap-Revenue by Store and Product Category")
fig.show()
```

12.Product Price vs Units Sold(Hexbinn Plot)

In [84]:
```
hex_df=merged_df[['Product_Price','Units']].dropna()
hex_df=hex_df[(hex_df['Product_Price']>0)&(hex_df['Units']>0)]
plt.figure(figsize=(12,8))
hb=plt.hexbin(hex_df['Product_Price'],hex_df['Units'],gridsize=30,cmap='plasma',
plt.colorbar(hb,label='Number of Transactions')
plt.xlabel('ProductPrice')
plt.ylabel('UnitsSold')
plt.title('Hexbinn Plot-Product Price vs Units Sold')
plt.show()
```

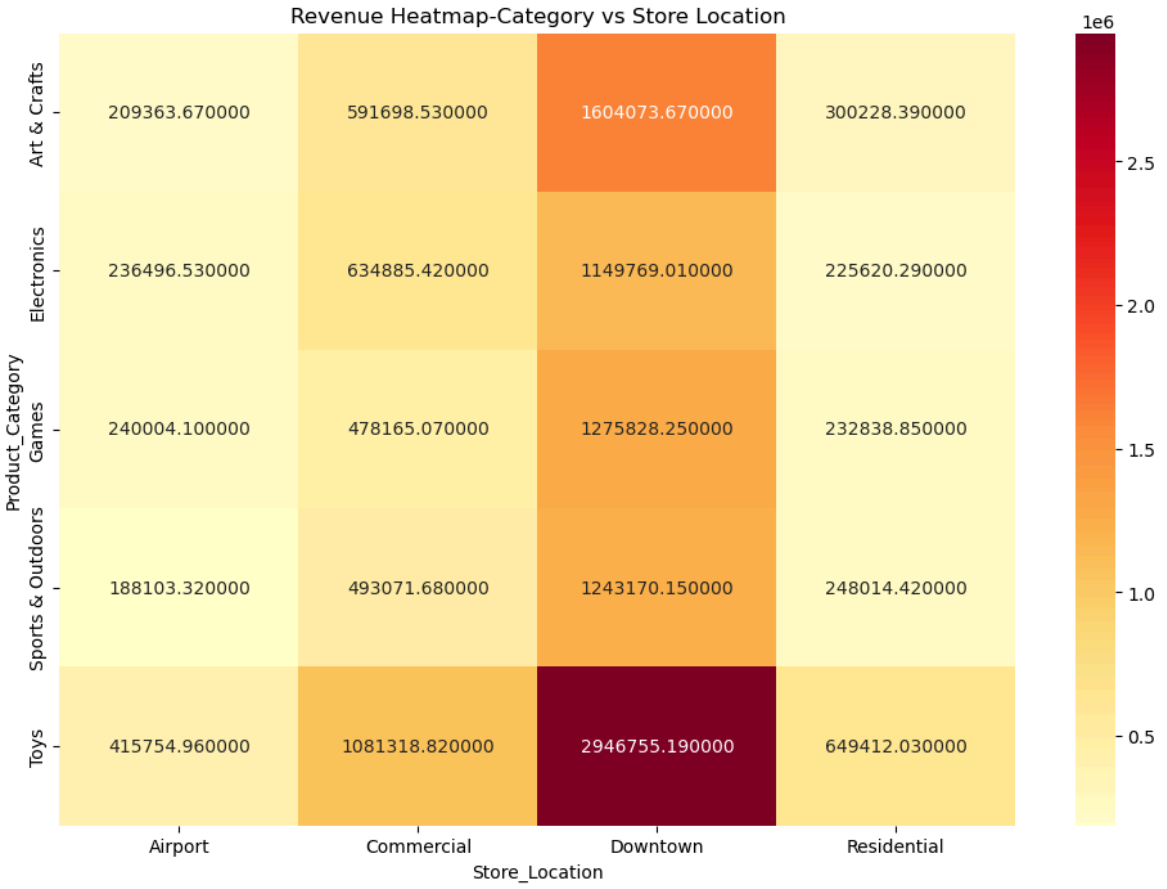


13.SunBurst Chart

In [85]:
```
sunburst_df=(merged_df.groupby(['Product_Category','Product_Name'],as_index=Fals
fig=px.sunburst(sunburst_df,path=['Product_Category','Product_Name'],values='Rev
                title='Revenue Hierarchy-Category to Product')
fig.update_layout(width=1000,height=750)
fig.show()
```

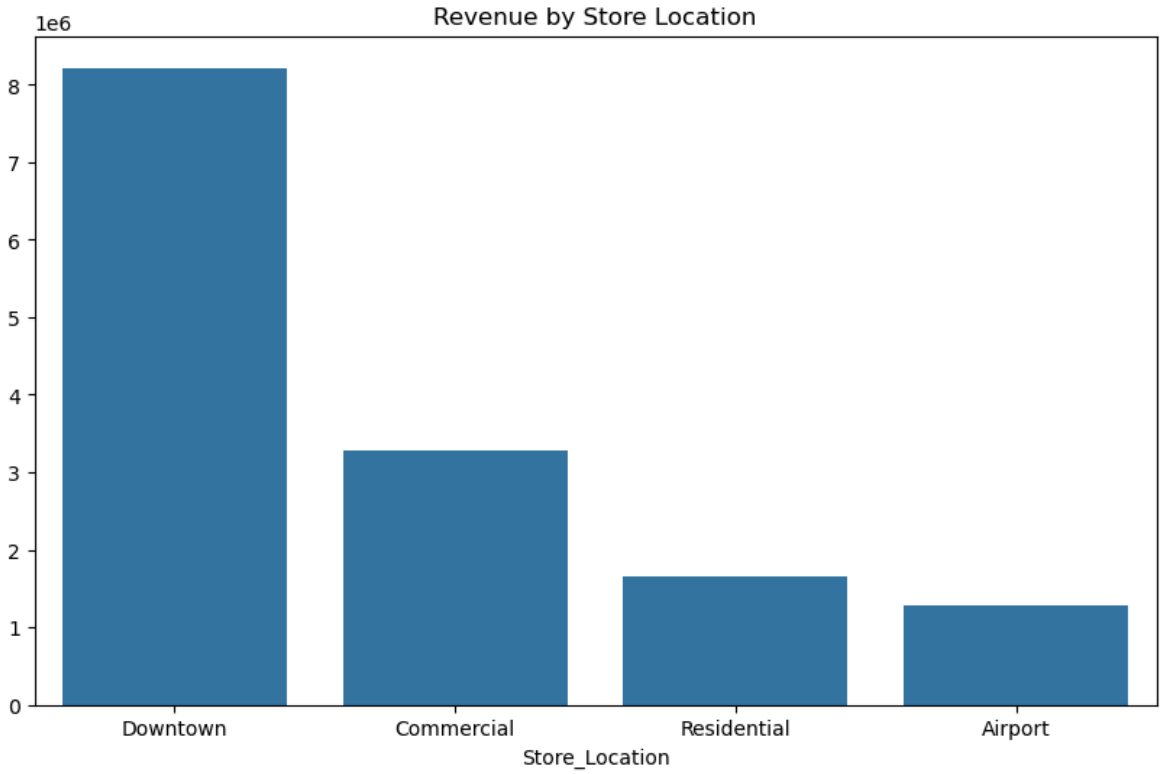
14.Category-Store(Heatmap)

In [86]:
```
heatmap_data=pd.pivot_table(merged_df,values='Revenue',index='Product_Category',
plt.figure(figsize=(12,8))
sns.heatmap(heatmap_data,annot=True,fmt='0f',cmap='YlOrRd')
plt.title('Revenue Heatmap-Category vs Store Location')
plt.show()
```



15.Store Location Analysis

In [87]:
```
location_sales=(merged_df.groupby('Store_Location')['Revenue'].sum().sort_values
plt.figure(figsize=(10,6))
sns.barplot(x=location_sales.index,
            y=location_sales.values)
plt.title('Revenue by Store Location')
plt.show()
```


Revenue by Store Location
1e6
8
7
6
5
4
3
2
1
0
Downtown
Commercial
Residential
Airport
Store_Location

In [ ]: